Napredne veb tehnologije – Specifikacija ispitnog zadatka

Softversko inženjerstvo i informacione tehnologije 2025/2026

1. O predmetu **2**
1.1. Raspored vežbi 2
2. O projektu **3**
3. Uloge korisnika **4**
4. "Pametna" kompanija **4**
4.1. Proizvodni pogon (fabrika) 4
4.2. Magacin 5
4.3. Dostavno vozilo 5
5. Funkcionalni zahtevi **5**
5.1. Inicijalno stanje sistema i upravljanje menadžera (Student 1) 5
5.2. Upravljanje proizvodima (Student 1) 6
5.3. Registracija i prijava kupca (Student 2) 6
5.4. Kreiranje zahteva za registraciju firme (Student 3) 6
5.5. Obrada zahteva za registraciju firme (Student 3) 6
5.6. Kreiranje porudžbine (Student 2) 6
5.7. Upravljanje fabrikama (Student 1) 7
5.8. Upravljanje magacinima (Student 2) 7
5.9. Upravljanje dostavnim vozilima (Student 3) 7
5.10. Pregled rada fabrike (Student 1) 8
5.11. Pregled rada magacina (Student 2) 8
5.12. Pregled rada dostavnih vozila (Student 3) 8
5.13. Pregled dostupnosti fabrike (simulatora) (Student 1 - Bonus) 9
5.14. Pregled dostupnosti magacina (simulatora) (Student 2 - Bonus) 9
5.15. Pregled dostupnosti dostavnih vozila (simulatora) (Student 3 - Bonus) 9
6. Nefunkcionalni zahtevi **10**
6.1. Serverske platforme 10
6.2. Klijentske tehnologije 10
6.3. Slanje email-a 10
6.4. API dizajn 11
6.5. Perzistencija podataka 11
6.6. Serviranje statičkog sadržaja 11
6.7. Reverse proxy 11

6.8. Čuvanje korisničkih datoteka 11
6.9. Prikaz grafikona 12
6.10. Priprema podataka za testiranje i završnu odbranu 12
6.11. Testiranje performansi sistema 12
6.12. Kontrola verzija i kontinualni rad 13
7. Ocenjivanje 13
7.1. Bodovanje - opšte 13
7.2. Podela zadataka u timu 14
Student 1 14
Student 2 14
Student 3 14
7.3. Bonus zahtev 14
7.4. Finalna odbrana 14
7.4.1. Pravila finalne odbrana 15
8. Akademska čestitost 16

1. O predmetu

Cilj predmeta je osposobljavanje studenata za rešavanje problema iz oblasti Veb programiranja u infrastrukturnim sistemima. Podrazumeva proširenje znanja koje su student stekli na predmetima prethodne godine studija i upoznavanje studenata sa konceptima, strukturom i načinima razvoja modernih “bogatih” internet aplikacija (Rich Internet Applications – RIA). Prethodno uključuje poznavanje HTTP protokola, detalja serverskog sloja, osnova klijentskog sloja, osnove programskih okruženja, organizaciju veb aplikacija, detalje MVC arhitekture, osnove REST arhitekture, pregled infrastruktura za postavljanje veb aplikacija sa akcentom na one koje se koriste u infrastrukturnim sistemima, postavljanje aplikacija na single server i na cloud infrastrukture, kao i bezbednosne aspekte Veb programiranja za navedene infrastrukture, koje su nezaobilazni za infrastrukturne sisteme.

1.1. Raspored vežbi

Nedelja semestra	Datum prvog dana nedelje	Opis vežbe
1.	03.11.2025.	Prva nedelja - nema vežbi
2.	10.11.2025.	Druga nedelja - nema vežbi; **Prezentacija projekta u terminu predavanja**
3.	17.11.2025.	RabbitMQ
4.	24.11.2025.	InfluxDB

5.	01.12.2025.	Deployment; Reverse Proxy (Nginx)
6.	08.12.2025.	UI/UX
7.	15.12.2025.	Transakcije
8.	22.12.2025.	Keširanje podataka
9.	29.12.2025.	Konsultativni termin vežbi uz prethodnu najavu putem email-a*
10.	05.01.2026.	Konsultativni termin vežbi uz prethodnu najavu putem email-a*
11.	12.01.2026.	Locust
12.	19.01.2026.	Nedelja za individualni rad na projektu; Konsultativni termin vežbi uz prethodnu najavu putem email-a*
13.	26.01.2026	Nedelja za individualni rad na projektu; Konsultativni termin vežbi uz prethodnu najavu putem email-a*
14.	02.02.2026.	Nedelja za individualni rad na projektu; Konsultativni termin vežbi uz prethodnu najavu putem email-a*
15.	09.02.2026.	**Odbrana projekta - prvi rok**

*Ako se nijedan student iz grupe ne najavi putem email-a, vežbe u redovnom terminu neće biti održane.

2. O projektu

Vaš zadatak je da projektujete i implementirate softversko rešenje za rad kompanije koja proizvodi i prodaje veliki broj proizvoda (veleprodaja). Kompanija ima veliki broj proizvodnih pogona (fabrika), magacina i dostavnih vozila u mnogim državama širom Evrope. Želja uprave kompanije jeste da postoji jedna platforma u oblaku koja bi bila dostupna putem interneta i kojoj bi pristupali zaposleni širom Evrope. Platformu bi zaposleni koristili za praćenje proizvodnje i distribucije proizvoda. Istu platformu bi takođe koristile i kompanije širom Evrope za poručivanje velikih količina proizvoda koji su u ponudi.

3. Uloge korisnika

Platforma razlikuje sledeće vrste korisnika koji koriste aplikaciju:

- Menadžeri: zaposleni kompanije koja je vlasnik platforme, čiji je zadatak upravljanje procesima distribucije i prodaje proizvoda;

- Kupci: vlasnici registrovanih firmi koji mogu da poručuju proizvode;
- Neautentifikovani korisnici: imaju mogućnost registracije kao novi kupci i prijave na sistem.

4. "Pametna" kompanija

Po uzoru na koncepte pametnih gradova i pametnih kuća, kompanija se takođe da bude "pametna" te je cilj da se putem platforme nadziru i upravljaju sledeći objekti:

- Proizvodni pogoni (fabrike);
- Magacini;
- Dostavna vozila.

Sva tri tipa objekata imaju "pametni" kontroler (računar ili uređaj) koji prikuplja informacije objekta i šalje ih platformi, a u zavisnosti od tipa objekta se putem platforme može upravljati objektom. Rad sva tri tipa objekta potrebno je simulirati pomoću malih i jednostavnih aplikacija (npr. konzolnih) koje imaju ulogu **simulatora** (aplikacije simuliraju ponašanje fizičkih sistema).

Prilikom implementacije simulatora potrebno je ispuniti sledeće zahteve:

1. Simulator periodično javlja platformi da je online (*heartbeat*) - u slučaju da se neko duže vreme nije javio da je online (npr. 60 sekundi), platforma treba da smatra da više nije na mreži i to treba da zabeleži u bazi podataka;
2. Simulator komunicira sa platformom putem AMQP protokola.

4.1. Proizvodni pogon (fabrika)

Fabrika je objekat koji proizvodi jedan ili više različitih tipova proizvoda. Potrebno je **napisati simulator fabrike** koji periodično, 2 puta dnevno, simulira "proizvodnju" proizvoda i javlja platformi koji broj proizvoda nekog tipa je proizvedeno. Na primer, u podne simulator javi da je proizvedeno 100 guma tipa 195/55/R15-H, 200 guma tipa 225/65/R17-W i 300 guma tipa 205/60/R16-Y, a u ponoć javi da je proizvedeno 300 guma tipa 195/55/R15-H, 100 guma tipa 225/65/R17-W i 400 guma tipa 205/60/R16-Y. Prilikom pokretanja simulatora definisati (kroz komandnu liniju ili konfiguracionu datoteku) koje proizvode fabrika proizvodi kao i podatke koji su potrebni kako bi platforma znala sa kojom fabrikom komunicira.

4.2. Magacin

Magacin je objekat u kom se čuvaju proizvedeni proizvodi. Magacin ima više sektora u kojima su drugačiji klimatski uslovi koji su prilagođeni proizvodima koji se čuvaju u njima. Potrebno je **napisati simulator magacina** koji periodično, na svakih 10 minuta, za svaki sektor simulira temperaturu i javlja te podatke platformi. Prilikom pokretanja simulatora definisati (kroz

komandnu liniju ili konfiguracionu datoteku) podatke o sektorima magacina kao i podatke koji su potrebni kako bi platforma znala sa kojim magacinom komunicira.

4.3. Dostavno vozilo

Dostavno vozilo je prevozno sredstvo kojim se prenose proizvodi od magacina do kranjih kupaca. Svako vozilo osim standardnih podataka o prevoznom sredstvu ima i ograničenje težine koju može da prevozi. Potrebno je **napisati simulator dostavnog vozila** koji periodično, na svakih 10 minuta, javlja platformi koja je lokacija vozila (geografska širina i dužina) i koliko je kilometara prešlo u međuvremenu između dva javljanja. Prilikom pokretanja simulatora definisati (kroz komandnu liniju ili konfiguracionu datoteku) podatke o vozilu kao i podatke koji su potrebni kako bi platforma znala sa kojim vozilom komunicira.

5. Funkcionalni zahtevi

5.1. Inicijalno stanje sistema i upravljanje menadžera (Student 1)

Kada se softversko rešenje pokrene, potrebno je automatski (prilikom prvog podizanja aplikacije) generisati supermenadžer nalog (sa username-om **admin**). Predefinisana lozinka supermenadžera treba da bude nasumično generisana tekstualna vrednost koja sadrži veći broj karaktera. Predefinisanu lozinku u izvornom obliku sačuvati u tekstualnu datoteku, a lokaciju ove datoteke navesti u uputstvu za pokretanje aplikacije. Supermenadžer (kao i običan menadžer) može da se prijavi na sistem. Nakon prve prijave, potrebno je zahtevati od supermenadžera promenu predefinisane lozinke i ne dozvoliti mu pristup aplikaciji dok to ne uradi.
Supermenadžer ima sve iste funkcionalnosti kao obični menadžeri. Supermenadžer može da registruje nove menadžere. Nakon što popuni formu za registraciju (u sklopu koje upload-uje profilnu sliku) i uspešno izvrši registraciju, nalog novokreiranog menadžera je automatski aktivan. Nakon prve prijave, potrebno je zahtevati od novokreiranog menadžera promenu predefinisane lozinke i ne dozvoliti mu pristup aplikaciji dok to ne uradi. Supermenadžer može da vidi spisak menadžera i da blokira nekog menadžera nakon čega on ne može više da se prijavi na sistem. Implementirati efikasan mehanizam prikaza i pretrage menadžera po uzoru na društeve mreže.

5.2. Upravljanje proizvodima (Student 1)

Menadžer može da uređuje (CRUD) sve proizvode koje kompanija proizvodi i nudi. Svaki proizvod pored osnovnih informacija ima i sliku, cenu i težinu. Proizvodi su organizovani u kategorije. Spisak proizvoda može da se pretraži. Implementirati efikasan mehanizam pretrage proizvoda po uzoru na vebšopove. Za svaki proizvod može da se naznači da li je na prodaju

(npr. neki proizvod se naznači da nije na prodaju dok se ne stvore određene zalihe). Za svaki proizvod može da se naznači u kojim fabrikama se proizvodi.

5.3. Registracija i prijava kupca (Student 2)

Dizajnirati početnu stranu koju vidi neprijavljeni korisnik sa linkovima na stranice za registraciju i prijavu. Omogućiti registraciju i prijavu novih kupaca na sistem. Nakon što korisnik popuni formu za registraciju (u sklopu koje upload-uje svoju profilnu sliku) i uspešno se registruje, njegov nalog nije aktivan, već sistem šalje email na unetu adresu korisnika sa linkom za aktivaciju naloga. Nakon što se nalog aktivira, korisnik može da se prijavi. Prijavljeni korisnik u bilo kom trenutku može da se odjavi sa sistema.

5.4. Kreiranje zahteva za registraciju firme (Student 3)

Kupac koji se registrovao kroz funkcionalnost 6.2. može da kreira jednu ili više firmi. Za svaku firmu koju kreira mora da preda zahtev za registraciju firme. Pored osnovnih informacija (poput naziva, adrese, države, grada), jedne ili više slika kompanije (koje se upload-uju), mora da navede lokaciju tako što je označi na klikom na mapi (koristiti bilo koje dostupno rešenje: npr. OpenLayers map, Leaflet, Modest maps, Polymaps itd.). Za izbor države i grada ponuditi izbor iz predefinisanih spiskova država i gradova koji se nalaze u bazi podataka. Takođe, potrebno je da priloži jednu ili više slika i/ili jedan ili više PDF dokumenata kojima dokazuje da ima pravo da registruje firmu (dokaz o vlasništvu i drugi dokumenti).

5.5. Obrada zahteva za registraciju firme (Student 3)

Menadžer može da obradi zahtev za registraciju firme. Tom prilikom pregleda sve informacije i dokumente koji su priloženi na osnovu čega donosi odluku da odobrava i odbija zahtev. Ako odbije zahtev neophodno je da navede obrazloženje zašto se odbija. Odluka menadžera šalje se kupcu na email.

5.6. Kreiranje porudžbine (Student 2)

Kupac može da kreira porudžbinu. Od ponuđenih proizvoda bira one koje želi da kupi u količinama koje su mu potrebne. Izabrana količina na sme biti veće od sume količine izabranih proizvoda u svim magacinima. Implementirati efikasan mehanizam pretrage proizvoda po uzoru na vebšopove. Takođe, bira i firmu u koju želi da proizvodi budu dostavljeni. Ako je porudžbina uspešno kreirana (ima dovoljno poručenih proizvoda na stanju, uspešno se kreira porudžbina i smanji se količina poručenih proizvoda) kreira se PDF faktura koja se šalje kupcu na email.

5.7. Upravljanje fabrikama (Student 1)

Menadžer može da upravlja fabrikama (CRUD). Može da kreira, izmeni i obriše (fizički) informacije fabrike. Prilikom kreiranja fabrike, pored osnovnih informacija (poput naziva, adrese, države, grada.), jedne ili više slika fabrike (koje se upload-uju), mora da navede lokaciju tako što

je označi na klikom na mapi (koristiti bilo koje dostupno rešenje: npr. OpenLayers map, Leaflet, Modest maps, Polymaps itd.). Za izbor države i grada ponuditi izbor iz predefinisanih spiskova država i gradova koji se nalaze u bazi podataka. U sklopu pregleda spiska fabrika implementirati efikasan mehanizam za pretraživanje fabrika.
Implementirati simulator za fabriku prema zahtevima iz poglavlja 4. Fabrika će periodično javljati informacije o proizvodnji proizvoda što je neophodno sačuvati u relacionoj bazi podataka. Podatke o dostupnosti simulatora, promeni stanja simulatora i količini simuliranih proizvoda čuvati u *time series* bazi podataka. Prikazati kada je fabrika "na mreži", odnosno kada nije na osnovu *heartbeat*-a.

5.8. Upravljanje magacinima (Student 2)

Menadžer može da upravlja magacinima (CRUD). Može da kreira, izmeni i obriše (fizički) informacije magacina. Prilikom kreiranja magacina, pored osnovnih informacija (poput naziva, adrese, države, grada.), jedne ili više slika magacina (koje se upload-uju), mora da navede lokaciju tako što je označi na klikom na mapi (koristiti bilo koje dostupno rešenje: npr. OpenLayers map, Leaflet, Modest maps, Polymaps itd.). Za izbor države i grada ponuditi izbor iz predefinisanih spiskova država i gradova koji se nalaze u bazi podataka. U sklopu pregleda spiska magacina implementirati efikasan mehanizam za pretraživanje magacina.
Implementirati simulator za magacin prema zahtevima iz poglavlja 4. Magacin će periodično javljati informacije o vremenskim uslovima za svaki sektor. Poslednju vrednost temperature za svaki sektor sačuvati u relacionoj bazi podataka. Podatke o dostupnosti simulatora, promeni stanja simulatora i sva temperaturna merenja svih sektora čuvati u *time series* bazi podataka. Prikazati kada je magacin "na mreži", odnosno kada nije na osnovu *heartbeat*-a.

5.9. Upravljanje dostavnim vozilima (Student 3)

Menadžer može da upravlja dostavnim vozilima (CRUD). Može da kreira, izmeni i obriše (fizički) informacije vozila. Prilikom kreiranja vozila, pored osnovnih informacija (poput registarske oznake, ograničenja težine, marke i modela), upload-uje i jednu ili više slika vozila (koje se upload-uju). Za izbor marke i modela vozila ponuditi izbor iz predefinisanih spiskova marki i modela koji se nalaze u bazi podataka (po uzoru na aplikacije za prodaju novih i polovnih vozila). U sklopu pregleda spiska vozila implementirati efikasan mehanizam za pretraživanje vozila.
Implementirati simulator za dostavno vozilo prema zahtevima iz poglavlja 4. Dostavno vozilo će periodično javljati informacije o lokaciji i pređenoj distanci. Poslednju vrednost lokacije sačuvati u relacionoj bazi podataka. Podatke o dostupnosti simulatora, promeni stanja simulatora i podatke o pređenim distancama čuvati u *time series* bazi podataka. Prikazati kada je dostavno vozilo "na mreži", odnosno kada nije na osnovu *heartbeat*-a.

5.10. Pregled rada fabrike (Student 1)

Menadžer može da pretraži sve fabrike u sistemu. Implementirati efikasan mehanizam pretrage fabrika. Izborom fabrike može da vidi sve njene detalje. Za izabranu fabriku može da vidi sve

proizvode koji se proizvode u njoj. Za izabrani proizvod može videti grafike ukupne proizvodnje. Menadžer može da izabere vremenski period u kom želi da vidi ukupnu količinu proizvedenih jedinica proizvoda za: poslednjih nedelju dana, mesec dana, 3 meseca, 6 meseci, godinu dana (fiksan izbor jedne od prethodno navedenih opcija) ili da izabere datume od-do pri čemu razlika između ta dva datuma ne sme da bude veća od godinu dana. Za izabrani vremenski period, korisnik vidi koliko je proizvedeno jedinica za izabrani proizvod. U slučaju da je u pitanju kraći vremenski period (npr. poslednih nedelju dana), prikazati izračunatu vrednost za svaki dan, dok ako je izabran duži vremenski period (npr. 6 meseci), prikazati izračunatu vrednost unutar svake nedelje ili meseca.

5.11. Pregled rada magacina (Student 2)

Menadžer može da pretraži sve magacine u sistemu. Implementirati efikasan mehanizam pretrage magacina. Izborom magacina može da vidi sve njegove detalje. Za izabrani magacin može da vidi sve sektore i da za izabrani sektor vidi poslednju očitanu temperaturu. Takođe za izabrani sektor može videti grafike prosečne temperature. Menadžer može da izabere vremenski period u kom želi da vidi prosečnu temperaturu za: poslednjih nedelju dana, mesec dana, 3 meseca, 6 meseci, godinu dana (fiksan izbor jedne od prethodno navedenih opcija) ili da izabere datume od-do pri čemu razlika između ta dva datuma ne sme da bude veća od godinu dana. Za izabrani vremenski period, korisnik vidi prosečnu temperaturu za izabrani sektor. U slučaju da je u pitanju kraći vremenski period (npr. poslednih nedelju dana), prikazati izračunatu vrednost za svaki dan, dok ako je izabran duži vremenski period (npr. 6 meseci), prikazati izračunatu vrednost unutar svake nedelje ili meseca.

5.12. Pregled rada dostavnih vozila (Student 3)

Menadžer može da pretraži sva dostavna vozila u sistemu. Implementirati efikasan mehanizam pretrage vozila. Izborom vozila može da vidi sve njegove detalje. Za izabrano vozilo može da vidi vidi poslednju očitanu lokaciju. Takođe za izabrano vozilo može videti grafike ukupne pređene distance. Menadžer može da izabere vremenski period u kom želi da vidi ukupnu pređenu distancu za: poslednjih nedelju dana, mesec dana, 3 meseca, 6 meseci, godinu dana (fiksan izbor jedne od prethodno navedenih opcija) ili da izabere datume od-do pri čemu razlika između ta dva datuma ne sme da bude veća od godinu dana. Za izabrani vremenski period, korisnik vidi ukupnu pređenu distancu za izabrano dostavno vozilo. U slučaju da je u pitanju kraći vremenski period (npr. poslednih nedelju dana), prikazati izračunatu vrednost za svaki dan, dok ako je izabran duži vremenski period (npr. 6 meseci), prikazati izračunatu vrednost unutar svake nedelje ili meseca.

5.13. Pregled dostupnosti fabrike (simulatora) (Student 1 - Bonus)

Menadžer može da pretraži sve fabrike u sistemu. Implementirati efikasan mehanizam pretrage fabrika. Izborom fabrike može da vidi sve njene detalje. Za izabranu fabriku može da vidi grafike

dostupnosti. Menadžer može da izabere vremenski period u kom želi da vidi dostupnost simulatora za: poslednji 1h, poslednih 12h, poslednja 24h, poslednjih nedelju dana, mesec dana, 3 meseca, godinu dana (fiksan izbor jedne od prethodno navedenih opcija) ili da izabere datume od-do pri čemu razlika između ta dva datuma ne sme da bude veća od godinu dana. Za izabrani vremenski period, korisnik vidi koliko je procentualno i vremenski simulator bio na mreži i koliko nije bio na mreži (koliko je bio online, a koliko offline). U slučaju da je u pitanju kraći vremenski period (npr. poslednih 12h), prikazati dostupnost simulatora unutar svakog sata, dok ako je izabran duži vremenski period (npr. 1 mesec), prikazati dostupnost simulatora unutar svakog dana ili nedelje. Dostupnost simulatora prikazati pomoću različitih grafika (bar chart, plot-ovi itd.). U slučaju da je izabran period od poslednjih 3 sata, omogućiti prikaz u realnom vremenu. Dobavljanje podataka radi prikaza akcija u realnom vremenu ostvariti pomoću WebSocket protokola.

5.14. Pregled dostupnosti magacina (simulatora) (Student 2 - Bonus)

Menadžer može da pretraži sve magacine u sistemu. Implementirati efikasan mehanizam pretrage magacina. Izborom magacina može da vidi sve njegove detalje. Za izabrani magacin može da vidi grafike dostupnosti. Menadžer može da izabere vremenski period u kom želi da vidi dostupnost simulatora za: poslednji 1h, poslednih 12h, poslednja 24h, poslednjih nedelju dana, mesec dana, 3 meseca, godinu dana (fiksan izbor jedne od prethodno navedenih opcija) ili da izabere datume od-do pri čemu razlika između ta dva datuma ne sme da bude veća od godinu dana. Za izabrani vremenski period, korisnik vidi koliko je procentualno i vremenski simulator bio na mreži i koliko nije bio na mreži (koliko je bio online, a koliko offline). U slučaju da je u pitanju kraći vremenski period (npr. poslednih 12h), prikazati dostupnost simulatora unutar svakog sata, dok ako je izabran duži vremenski period (npr. 1 mesec), prikazati dostupnost simulatora unutar svakog dana ili nedelje. Dostupnost simulatora prikazati pomoću različitih grafika (bar chart, plot-ovi itd.). U slučaju da je izabran period od poslednjih 3 sata, omogućiti prikaz u realnom vremenu. Dobavljanje podataka radi prikaza akcija u realnom vremenu ostvariti pomoću WebSocket protokola.

5.15. Pregled dostupnosti dostavnih vozila (simulatora) (Student 3 - Bonus)

Menadžer može da pretraži sva dostavna vozila u sistemu. Implementirati efikasan mehanizam pretrage dostavnih vozila. Izborom dostavnog vozila može da vidi sve njegove detalje. Za izabrano dostavno vozilo može da vidi grafike dostupnosti. Menadžer može da izabere vremenski period u kom želi da vidi dostupnost simulatora za: poslednji 1h, poslednih 12h, poslednja 24h, poslednjih nedelju dana, mesec dana, 3 meseca, godinu dana (fiksan izbor jedne od prethodno navedenih opcija) ili da izabere datume od-do pri čemu razlika između ta dva datuma ne sme da bude veća od godinu dana. Za izabrani vremenski period, korisnik vidi koliko je procentualno i vremenski simulator bio na mreži i koliko nije bio na mreži (koliko je bio online, a koliko offline). U slučaju da je u pitanju kraći vremenski period (npr. poslednih 12h),

prikazati dostupnost simulatora unutar svakog sata, dok ako je izabran duži vremenski period (npr. 1 mesec), prikazati dostupnost simulatora unutar svakog dana ili nedelje. Dostupnost simulatora prikazati pomoću različitih grafika (bar chart, plot-ovi itd.). U slučaju da je izabran period od poslednjih 3 sata, omogućiti prikaz u realnom vremenu. Dobavljanje podataka radi prikaza akcija u realnom vremenu ostvariti pomoću WebSocket protokola.

6. Nefunkcionalni zahtevi

6.1. Serverske platforme

Za realizaciju projekta može se izabrati serverska platforma po želji. Neke od platformi mogu biti:

- Java + Spring Boot (koristi se na vežbama)
- Java + Play framework
- Java + Spark framework
- Python + Django
- Python + Flask
- Python + FastAPI
- Ruby on Rails
- .NET
- …

6.2. Klijentske tehnologije

Za realizaciju klijentske aplikacije može se izabrati jedan od sledećih frontend tehnologija:

- Angular
- Vue
- React

6.3. Slanje email-a

Za slanje email-a nije obezbeđen poseban servis. Možete koristiti sopstveni email nalog. Email-ovi koje šalje vaša platforma treba da imaju **jasno definisan template koji izgledom podseća na izgled vaše platforme**.

6.4. API dizajn

API glavne aplikacije treba da bude dizajniran i dokumentovan u skladu sa OpenAPI specifikacijom.

6.5. Perzistencija podataka

Sve podatke koji se kreiraju u toku rada platforme potrebno je trajno perzistirati. Podatke koje generiše simulator (poput merenja i promene stanja/statusa) potrebno je čuvati u time series bazi podataka po izboru (npr. InfluxDb). Ostale podatke (poput informacija o korisnicima, objektima itd.) potrebno je čuvati u relacionoj bazi podataka po izboru (npr. MySQL, MariaDB, PostgreSQL itd.).

Neophodno je primeniti tehnike i implementirati mehanizme za očuvanje konzistentnosti podataka u slučaju da se desi neočekivana greška prilikom čuvanja nekog podatka ili ako se istovremeno menja isti podatak za sledeće funkcionalnosti:

- Student 1 - 5.2. i 5.7.
- Student 2 - 5.6. i 5.8.
- Student 3 - 5.5. i 5.9.

6.6. Serviranje statičkog sadržaja

Iako mnoge serverske platforme imaju mogućnost da serviraju statički sadržaj i sadržaj koji korisnici upload-uju (poput slika), takav način serviranja sadržaja nije preporučljiv. Umesto toga, tehnologije poput nginx-a mogu da se koriste kao reverse proxy i da bi servirali statički sadržaj. Česta upotreba npr. nginx-a je serviranje statičkog sadržaja koji čini korisnički interfejs aplikacije (html, css, js itd.). U tom slučaju, ako je klijentska aplikacija izrađena pomoću nekog radnog okvira i/ili skupa biblioteka (npr. Angular, React, Vue), ona se pretvori u skup statičkog sadržaja (tj. izgradi za produkciju - *production build*) koji nginx servira.

6.7. Reverse proxy

Kada imate softversko rešenje koje ima više komponenti ili resursa koji su dostupni na različitim rutama, a pri čemu je potrebno definisati način pristupa, dobra praksa je koristiti reverse proxy (npr. nginx, Apache Traffic Server, HAProxy itd.). Reverse proxy može da reši i neke druge probleme sa kojima se možete susresti prilikom izrade projektnog zadatka.

6.8. Čuvanje korisničkih datoteka

Potrebno je obezbediti sigurno čuvanje datoteka koje korisnici upload-uju kao što su slike i dokumenti. Kako ove datoteke mogu biti izuzetno velike, ne bi trebalo da se njihovim serviranjem bavi serverska aplikacija. Stoga, potrebno je izabrati tehnologiju čiji će zadatak biti da omogući prikaz i preuzimanje ovih datoteka. Takođe, potrebno je obezbediti proveru prava pristupa ovim datotekama, tj. da ne može svako da preuzme svaku od datoteka već da se reguliše pravo pristupa u skladu sa specifikacijom funkcionalnih zahteva.

6.9. Prikaz grafikona

Za prikaz grafikona na korisničkom interfejsu mogu se koristiti third party biblioteke.

6.10. Priprema podataka za testiranje i završnu odbranu

Neophodno je u relacionoj bazi podataka imati sledeće podatke:

- minimalno 2 miliona kupaca; najmanje 1 kupac treba da ima najmanje 100 firmi;
- minimalno 2 miliona registrovanih i prihvaćenih firmi;
- minimalno 1 milion menadžera;
- minimalno 20 000 fabrika;
- minimalno 40 000 magacina
- minimalno 100 000 dostavnih vozila.

Neophodno je u *time series* bazi podataka imati sledeće podatke:

- za najmanje 1000 fabrika bi trebalo imati realne podatke proizvodnje proizvoda, pri čemu svaka fabrika pravi po najmanje 10 proizvoda, za prethodne 4 godine;
    - 1000 fabrika * 10 proizvoda * 4 godine * 365.25 dana u godini * 2 javljanja dnevno = 29 220 000 zapisa;
- za najmanje 100 magacina bi trebalo imati realne podatke merenja temperature, pri čemu svaki magacin ima najmanje 3 sektora, za prethodne 2 godine;
    - 100 magacina * 3 sektora * 2 godine * 365.25 dana u godini * 144 javljanja dnevno = 31 557 600 zapisa;
- za najmanje 100 dostavnih vozila bi trebalo imati realne podatke pređene distance između dva javljanja, za prethodneih 5 godina;
    - 100 vozila * 5 godine * 365.25 dana u godini * 144 javljanja dnevno = 26 298 000 zapisa.

6.11. Testiranje performansi sistema

Potrebno je izvršiti testiranje opterećenja sistema (*load testing*) za česte scenarije korišćenja. Svaki student treba da izabere 10 različitih endpoint-ova koje je implementirao u sklopu funkcionalnosti koje mu pripadaju. Izabrati najbitnije endpoint-ove koji pripadaju nakritičnijim funkcionalnostima, tj. onima za koje procenite da će vremenski biti najzahtevnije ili koje istovremeno treba da koristi veliki broj korisnika. Za svaki od endpoint-ova treba da se provere performanse sistema i ispita njihova promena sa povećanjem količine podataka sa kojim aplikacija rukuje i povećanjem broja korisnika koji istovremeno pokušavaju da izvrše isti scenario (npr. veliki broj korisnika istovremeno pokuša da se prijavi na sistem i kreira zahtev za registraciju nove nekretnine). Potrebno je istražiti koji alati se mogu koristiti za ovaj vid testiranja (JMeter, Locust, Gatling itd.) i koristiti jedan i/ili više njih kako bi se izvršilo potpuno testiranje izabranih scenarija korišćenja. Testirati ponašanje aplikacija počev od slučaja kada je istovremeno koristi mali broj korisnika (10-ak) do slučaja kada je koristi veliki broj korisnika (nekoliko hiljada).

Potrebno je da svaki student sastavi izveštaj (u slobodnoj formi) kako je izvršio testiranje svog dela sistema. Dokument može da sadrži tekst, slike i grafikone koji dokumentuju proces testiranja. U slučaju da prilikom testiranja uočite da određeni delovi sistema imaju loše performanse, potrebno je uložiti napore u unapređenje tih delova sistema i to bi trebalo takođe dokumentovati u izveštaju. Konačnu verziju izveštaja konvertovati u **PDF dokument** i okačiti na udaljeni git repozotorijum. Dokument treba da sadrži specifikaciju računara (ili više njih) na kom je pokrenuto rešenje i gde su pokretani testovi. Zatim, za svaki test opisati šta se testira i zašto, zabeležiti merenja i prokomentarisati dobijene rezultate. Napisati predlog kako se performanse za svaki od testiranih delova softvera mogu poboljšati i primeniti one predloge koje je moguće implementirati.

Napomena: Savetuje se da platformu (sa svim njenim komponentama) pokrenete na jednom računaru, a da na drugom računaru (ili više njih) pokrenete alate za testiranje performansi. Računari bi trebalo da se nalaze u istoj mreži kako bi mogli međusobno komunicirati. Na ovaj način ćete platformi dozvoliti da koristi što više resursa koje ima računar na kom je platforma pokrenuta, dok će drugi računar (ili više njih) služiti za pokretanje što više aplikacija koje će komunicirati sa platformom.

6.12. Kontrola verzija i kontinualni rad

Neophodno je koristiti **Git** za kontrolu verzija i repozitorijum mora biti na **GitLab**-u. Asistentskom nalogu Vanja-Mijatov (vanja.mijatov@uns.ac.rs) dati pristup repozitorijumu i ulogu **Maintainer**-a. Takođe, potrebno je u **README.md** napisati koje tehnologije su korišćene, ko je učestvovao u izradi rešenja i navesti tačno uputstvo za pokretanje projekta i priložiti skriptu za popunjavanje baze testnim podacima. Na odbrani projekta se brani isključivo kod koji se nalazi na **glavnoj grani** (*main* ili *master*). Ako timovi ne isprate ove korake pre odbrane projekta, rešenje neće moći biti pregledano (i testirano) pre odbrane od strane asistena te ni izlazak na odbranu neće biti moguć.

Napomena: Na GitLab-u će se kroz commit-ove pratiti kontinualni rad. Projekti koji budu kačeni na GitLab u svega nekoliko komitova neće biti podložni ocenjivanju. Nije neophodno raditi na projektu prema nekoj strogoj metodologiji (npr. Scrum) niti voditi tablu sa zadacima (kanban board ili nešto slično), ali je neophodno da se nove verzije koda šalju redovno u toku semestra na udaljeni repozitorijum.

7. Ocenjivanje

7.1. Bodovanje - opšte

Projekat ukupno nosi **50 bodova**. Minimalan broj bodova koji je potreban za polaganje projekta je **26**. Projekat se ocenjuje u toku odbrane projekta:

- Odbrana projekta - nosi 50 bodova i izvodi se u 3 roka:
    - Poslednja nedelja nastavnog dela zimskog semestra (druga nedelja februara);
    - Februarski rok (prva nedelja marta);
    - Septembarski rok (prva polovina septembra).

U sklopu odbrane projekta u istom terminu održaće se i usmeni ispit koji takođe nosi **50 bodova** kroz koji će se proveriti teorijsko znanje o implementiranim konceptima.

7.2. Podela zadataka u timu

Student 1

5.1, 5.2, 5.7, 5.10,

Student 2

5.3, 5.6, 5.8, 5.11,

Student 3

5.4, 5.5, 5.9, 5.12,

7.3. Bonus zahtev

Bonus zahtev predstavlja dodatni zahtev koji je moguće braniti isključivo ako se brani projekat u prvom ili drugom roku (februar i mart) i nosi 5 bodova. U slučaju da student izgubi određeni broj bodova (odnosno ne ostvari maksimalnih 50), izradom ove funkcionalnosti može da nadoknadi deo izgubljenih bodova (maksimalno 5). U trećem roku za odbranu projekta nije moguće ostvariti bodove za ovaj zahtev. Spisak bonus zahteva:

- Student 1 - 5.13.
- Student 2 - 5.14.
- Student 3 - 5.15.

7.4. Finalna odbrana

Implementacija svih funkcionalnosti je preduslov za uspešno polaganje projekta. Studenti koji ne implementiraju sve funkcionalnosti ne mogu uspešno položiti projekat. Da biste osvojili sve bodove za implementaciju neke funkcionalnosti nije bitno samo da li radi funkcionalnost, već i kako radi.

Kako aplikacija treba da podrži rad sa velikim brojem korisnika, objekata i simulatora, potrebno je da za svaku funkcionalnost nađete optimalno rešenje. To bi značilo da, ako je očekivan sporiji rad sistema ili njegovih delova zbog dobavljanja ili obrade veće količine podataka, koristeći neke od metoda obrade i keširanja podataka ubrzate rad sistema. Keširanje podataka se često radi na strani serverske aplikacije (obično uz korišćenje baze za keširanje poput redis-a). Svaki student bi trebao u sklopu funkcionalnosti koje mu pripadaju da identifikuje koji podaci mogu da

se efikasno keširaju te da implementira njihovo keširanje i osvežavanje keširanih podataka. Zatim, u slučaju prikazivanja veće količine podataka u vidu tabela ili grafika, potrebno je primeniti dobru praksu koja podrazumeva paginaciju, downsampling i slične pristupe. U tu svrhu dozvoljeno je koristiti gotova rešenja (aplikacije, biblioteke, dodatne baze podataka). Takođe, potrebno je obezbediti konzistentnost podataka i regulisati pristup istim u bazi podataka koristeći transakcije.

Drugi aspekt koji će se ocenjivati je priprema aplikacije za pokretanje u produkciji. Potrebno je:

- osmisliti arhitekturu sistema i isplanirati organizaciju redova poruka kako bi se podržala velika količina podataka te rešenje obrazložiti na odbrani;
- podesiti *reverse proxy* po izboru;
- neophodno je kreirati statičke datoteke na osnovu klijentske aplikacije i servirati ih korišćenjem neke tehnologije za serviranje statičkog sadržaja koja to efikasno radi;
- rešiti problem čuvanja i dobavljanja datoteka koje upload-uju korisnici;
- pojednostaviti konfiguraciju aplikacije te identifikovati minimalan set koraka i podešavanja koje bi bilo potrebno uraditi kako bi se rešenje pokrenulo u oblaku.

Korisničko iskustvo (UX) je takođe značajna stavka prilikom ocenjivanja projekta. Bitno je da klijentska aplikacija se intuitivno koristi i da akcije koje zahtevaju dobavljanje ili obradu veće količine podataka ne učine pretraživač da se zamrzne ili ugasi. Email-ovi koje šalje vaša platforma treba da imaju jasno definisan template koji izgledom podseća na izgled vaše platforme.

Na finalnoj odbrani će biti provereno ispunjavanje svih funkcionalnih i nefunkcionalnih zahteva sa akcentom na zahteve iz sekcija od 6.3. do 6.12. pri čemu je potrebno posebnu pažnju posvetiti zahtevima 6.10 i 6.11. Testiranje performansi sistema je izuzetan bitan aspekt u sklopu dizajniranja softvera koje će koristiti veliki broj korisnika te ovaj zahtev koji nosi najveći deo ocene.

Na finalnoj odbrani se brani verzija projekta koja se u trenutku isteka roka za predaju (tačni datum i vreme će za svaki rok biti istaknuti blagovremeno putem obaveštenja na Canvas platformi) nalazi na **glavnoj grani** (*main* ili *master*). Ako se tim prijavi za odbranu projekta, a do isteka roka za prijavu se asistentskom nalogu ne dozvoli pristup GitLab repozitorijumu (pri čemu asistentski nalog treba da ima ulogu *Maintainer*-a - pročitati poglavlje 6.12.), tim neće moći da brani projekat pošto projekat neće moći biti pregledan (i testiran) pre odbrane.

7.4.1. Pravila finalne odbrana

- Svaki tim bi trebao doći na odbranu najmanje 5 minuta pre definisanog termina u satnici kako bi pripremio okruženje za odbranu.
- Svaki tim pokreće tačno jednu instancu platforme na jednom ili više laptopova koji studenti treba sami da obezbede (u slučaju da se platforma sastoji od više aplikacija, moguće je pokrenuti ih na više različitih računara koji su spojeni u jednu mrežu).

- Na istom računaru/-ima potrebno je pokrenuti najmanje 2 instance simulatora za svaki tip simuliranih objekata.
- Za demonstraciju funkcionalnosti potrebno je ispuniti zahtev 6.10. U slučaju da ovaj zahtev nije ispunjen, neće biti moguće demonstriranje implementacije efikasnog rada rešenja za veliki broj slučajeva usled čega nije moguće ostvariti ocenu veću od 8.
- Prilikom demonstracije funkcionalnosti, studenti odgovaraju na pitanja o različitim tehnikama i pristupima koji su primenjeni na rešenju.
- Plagijarizam rešenja drugih timova nije dozvoljen (pročitati poglavlje *8. Akademska čestitost*).

8. Akademska čestitost

Akademska čestitost podrazumeva samostalno pisanje radova (teksta, programskog koda i slično) uz striktno poštovanje tuđih autorskih prava. Ovaj pojam i obaveza su sastavni deo Zakona o visokom obrazovanju:

http://www.parlament.gov.rs/upload/archive/files/lat/pdf/predlozi_zakona/3048-14 Lat.pdf

Više o ovoj temi možete pronaći, na primer, na sledećim linkovima:

- https://en.wikipedia.org/wiki/Academic_honor_code
- https://www.akademsko-pisanje.uniri.hr/akademska-cestitost/

U okviru ovog predmeta to podrazumeva da studenti samostalno pišu sopstveni rad. Pomoć drugih studenata u obliku direktno preuzetih delova teksta ili programa nije dozvoljena. **Kako se ovaj projekat radi u timu, odgovornost za nepridržavanje principa akademske čestitosti snose svi članovi tima.**

Sistemi za otkrivanje plagijarizma su vremenom postali prilično efektivni. Više informacija se može naći ovde:

https://theory.stanford.edu/~aiken/moss/

ili recimo ovde:

https://github.com/genchang1234/How-to-cheat-in-computer-science-101

Kako će se na ovom predmetu prilikom analize projekata pristupati originalnim udaljenim repozitorijumima, treba imati u vidu da se na njima vidi kompletna istorija izmena u toku rada na projektu. Okolnosti u kojima se na repozitorijumu pojavljuje nešto neregularno odmah dovode u sumnju dati studentski tim.

Zajedničko učenje studenata ili rad na projektu je lepa praksa, ali to ne podrazumeva da se do

rezultata koji se ocenjuje dolazi zajednički.